RoadVision: Real-Time Road Damage Detection using YOLO and Vision Transformers

Kiransarathi K
M.Sc. Artificial Intelligence & Machine Learning (III Year, Sem. V)
Department of Computing – AI & ML, Coimbatore Institute of Technology, Coimbatore – 641014, India
19MAM56 Advanced Machine Learning Lab – CAT II Project

October 13, 2026

Abstract

Manual road inspection is slow, subjective and expensive, while cracks and potholes grow quickly once water enters the pavement. We present *RoadVision*, an end-to-end system that detects four damage classes—longitudinal (D00), transverse (D10) and alligator (D20) cracks and potholes (D40)—in smartphone and dash-camera images of the multi-national RDD2022 dataset. The method is a two-stage cascade: a YOLO detector proposes candidate regions in real time, and a Vision Transformer (ViT) fine-tuned on context-padded crops, including a background class built by hard-negative mining, re-labels and re-scores each proposal. A fusion weight α and all decision thresholds are calibrated on the validation split only. Hyper-parameters are optimised with Bayesian search (Optuna TPE) and every run is tracked in MLflow. Under the CRDDC'2022 protocol (IoU>0.5, matching label) the hybrid achieves F1 = – (mAP50 –) at – FPS, against – for YOLO alone (paired bootstrap ΔF1 = –, 95% CI –). We benchmark against an RT-DETR transformer detector, a weighted-boxes-fusion ensemble and published challenge and journal results, report error taxonomy, size and country breakdowns, attention-rollout and Eigen-CAM explanations, and deploy the model as a validated FastAPI service with a Gradio UI in a Docker container on Hugging Face Spaces.

Index Terms— *road damage detection, YOLO, vision transformer, RT-DETR, RDD2022, hyper-parameter optimisation, explainable AI, MLOps*

1 Introduction

Road surface distress is the leading driver of road maintenance cost and a direct safety hazard for two-wheelers, which dominate Indian traffic. Visual surveys by engineers cover only a fraction of the network and are subjective. Because almost every vehicle now carries a camera, automatic damage detection from crowd-sensed images can turn every trip into a road survey [1, 2].

The problem is hard for an object detector: cracks are thin, elongated and low-contrast; crack subtypes differ by orientation and topology rather than texture; shadows, lane markings, manhole covers and repaired patches look like damage; and image statistics shift across countries and capture devices [4]. Challenge winners address this with large ensembles that are too slow for edge deployment [5, 3].

Objectives. (i) Build a reproducible data-engineering pipeline for RDD2022 with auditable cleaning, stratified splitting and augmentation; (ii) design a real-time detector that combines the speed of YOLO with the global context modelling of Vision Transformers; (iii) tune it automatically with Bayesian optimisation and track every experiment; (iv) benchmark against transformer detectors, ensembles and published state of the art with statistical rigour and error analysis; and (v) deploy it as a validated API and public web application.

Contributions. (1) A YOLO→ViT cascade with an explicit background class learned from mined false positives, and a two-parameter fusion rule calibrated on validation data; (2) a single evaluation harness that scores all models under the CRDDC protocol with bootstrap confidence intervals and a paired significance test; (3) a fully containerised deployment with input validation and a maintenance-priority severity index.

2 Literature Review

Datasets and challenges. Maeda *et al.* [1] released the first large smartphone road-damage dataset (Japan) and trained SSD detectors. It grew into RDD2020 (Japan, India, Czech Republic) for the IEEE BigData Global Road Damage Detection Challenge, whose winning YOLO ensemble reached F1 $\approx$ 0.67 / 0.66 on the two test sets [3, 2]. RDD2022 adds Norway, the USA and China (47,420 images, six countries) [4]. In CRDDC'2022 the best overall F1 was 0.770, obtained by an ensemble of YOLOv5/YOLOv7 with a Cascade R-CNN [10] using a Swin Transformer backbone [11]; the best single model (YOLOv7, third place) scored 0.741 [5]. Country-level F1 varied from 0.58 (India) to 0.84 (USA) for the same winning model, showing strong domain shift.

Detectors. One-stage YOLO detectors [7, 8] dominate the leaderboards because of their speed–accuracy trade-off. Two-stage detectors such as Faster R-CNN [9] are more accurate on small objects but slower. DETR [12] removed hand-crafted anchors and NMS through set prediction, and RT-DETR [13] made transformer detection real-time.

Vision Transformers. ViT [14] models an image as a sequence of patches with global self-attention, which suits long, thin structures that span many receptive fields; with ImageNet-21k pre-training and strong augmentation it transfers well to small datasets [15].

table `cleaning` – run the pipeline

Recent work. StripRFNet [6] (2025) adds strip receptive fields and shape-aware modules to a YOLO-style network and reports F1 = 0.816 on the Chinese subset of RDD2022, against 0.807 for YOLOv8s and 0.767 for RT-DETRv2-R18 under a random 8:1:1 split.

Research gaps. (G1) Leading results depend on heavy multi-model ensembles that are impractical in real time; (G2) most errors are confusions between crack subtypes and background false alarms, yet few works add a dedicated verification stage; (G3) results are rarely reported with confidence intervals, error taxonomies or explanations; (G4) few studies ship a deployable, validated system. RoadVision addresses G1–G4.

3 Methodology

3.1 System overview

Fig. 1 shows the pipeline. Raw VOC annotations are cleaned and split, YOLO and RT-DETR are trained on full images, a ViT is trained on crops, and the cascade is calibrated on validation data before a single test evaluation.

3.2 Data preprocessing pipeline

RDD2022 annotations are Pascal-VOC XML. We keep the four challenge classes and drop rare or out-of-scope labels (D43, D44, D50, Repair). Boxes are clipped to the image, boxes thinner than 4 px are removed as annotation noise, and exact duplicates are removed. Images without any damage (a form of missing labels) are not discarded but sub-sampled to 15% of the labelled count and kept as hard negatives. Corrupt images or XML files are skipped. Every step is counted (Table **??**). After cleaning, – images with – boxes remain. Norway (3650×2044 px, 9.9 GB) is excluded from the default run to fit a single 16 GB GPU.

The split is stratified on the joint key *country×dominant class*, so every country and class appears in train (–), validation (–) and test (–). Rare strata are merged into their country. Training uses Ultralytics mosaic, MixUp, HSV and geometric augmentation for the detectors, and an Albumentations [22] pipeline that simulates dash-cam conditions (motion blur, rain, fog, shadows, CLAHE) for the ViT.

3.3 Stage 1 – YOLO detector

YOLO (v8/11) predicts per anchor-free grid cell a box and class scores. Its loss is

$$\mathcal{L}_{\text{YOLO}} = \lambda_{\text{box}}\,\mathcal{L}_{\text{CIoU}} + \lambda_{\text{cls}}\,\mathcal{L}_{\text{BCE}} + \lambda_{\text{dfl}}\,\mathcal{L}_{\text{DFL}}, \tag{1}$$

where CIoU penalises overlap, centre distance and aspect-ratio mismatch, and the distribution focal loss (DFL) refines box edges. At inference we keep proposals with confidence ≥ 0.05 after NMS, deliberately favouring recall, because the ViT will remove false positives.

3.4 Stage 2 – Vision Transformer refiner

Each proposal b is enlarged by 15% context and resized to 224×224, then split into $N{=}196$ patches $x_p^i \in \mathbb{R}^{16\cdot16\cdot3}$:

$$z_0 = [x_{\text{cls}};\, x_p^1 E; \ldots; x_p^N E] + E_{\text{pos}}, \tag{2}$$

$$z'_\ell = \text{MSA}(\text{LN}(z_{\ell-1})) + z_{\ell-1}, \quad z_\ell = \text{MLP}(\text{LN}(z'_\ell)) + z \tag{3}$$

$$\text{Attn}(Q,K,V) = \text{softmax}\Big(QK^\top/\sqrt{d_k}\Big)\,V. \tag{4}$$

The class token z_L^0 is mapped to $K{+}1{=}5$ logits (D00, D10, D20, D40, BG). The BG class is trained on random road patches and on *hard negatives*: YOLO detections on the training split that match no ground truth (IoU<0.3). We fine-tune an ImageNet-21k ViT-S/16 [14, 16] with AdamW, a lower backbone learning rate (differential transfer learning), cosine schedule with warm-up, stochastic depth, label smoothing [17] and a class-balanced sampler:

$$\mathcal{L}_{\text{ViT}} = -\sum_{k=1}^{K+1} \big[(1-\varepsilon)\,y_k + \tfrac{\varepsilon}{K+1}\big] \log p_k. \tag{5}$$

3.5 Cascade fusion

For a proposal with YOLO label c_y, confidence s_y and ViT posterior p, let $\tilde{p}_k = p_k / \sum_{j\le K} p_j$ and $p_{\text{bg}} = p_{K+1}$. Then

$$\hat{c} = \arg\max_k \; \alpha \mathbb{1}[k = c_y] + (1-\alpha)\,\tilde{p}_k, \tag{6}$$

$$\hat{s} = s_y^{\alpha}\,\big[(1-p_{\text{bg}})\,\tilde{p}_{\hat{c}}\big]^{1-\alpha}. \tag{7}$$

$\alpha{=}1$ recovers YOLO, $\alpha{=}0$ trusts only the ViT. $\alpha \in \{0, 0.1, \ldots, 1\}$ and the score threshold are chosen by grid search to maximise validation F1 (selected α = –).

3.6 Transformer detector and ensemble baselines

RT-DETR [13] (ResNet/HGNet backbone, hybrid encoder, IoU-aware query selection) is trained with the Hungarian set-prediction loss $\mathcal{L} = \sum_i [\mathcal{L}_{\text{cls}} + \lambda_1 \|b_i - \hat{b}_{\sigma(i)}\|_1 + \lambda_2 \mathcal{L}_{\text{GIoU}}]$. We also fuse YOLO and RT-DETR outputs with Weighted Boxes Fusion [18], which averages the coordinates of overlapping boxes weighted by confidence×model weight and penalises boxes that only one model predicts.

3.7 Hyper-parameter search strategy

We use Optuna [19] with the multivariate Tree-structured Parzen Estimator [20], which models $\ell(x) = p(x \mid y{>}y^*)$ and $g(x) = p(x \mid y{\le}y^*)$ and proposes the x that maximises $\ell(x)/g(x)$ (equivalent to maximising expected improvement). A median pruner stops trials whose intermediate score falls below the median of earlier trials at the same epoch. YOLO search space: lr_0 (log, 10^{-4}–$2{\times}10^{-2}$), momentum, weight decay, optimiser {SGD, AdamW}, mosaic, MixUp, HSV value gain, rotation, box and class loss gains; objective is validation mAP50 after 15 epochs on 35% of the training data. ViT space: learning rate, backbone LR multiplier, weight decay, label smoothing and drop-path; objective is validation macro-F1. Studies are stored in SQLite (resumable), and every trial is a nested MLflow [21] run with parameters, per-epoch metrics and artefacts.

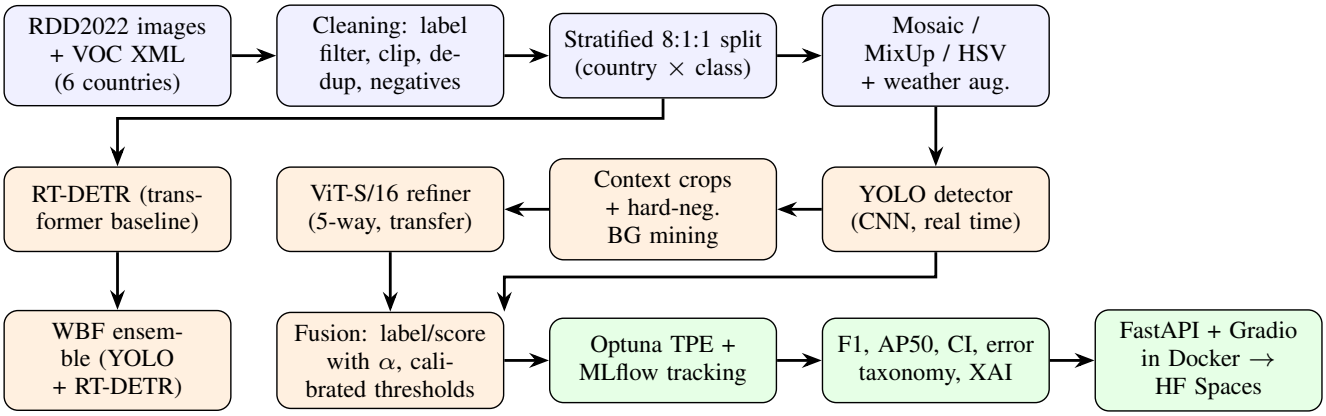


Figure 1: RoadVision block diagram: data engineering (blue), modelling (orange), optimisation, evaluation and deployment (green).

3.8 Evaluation protocol

Following CRDDC'2022 [5], a prediction is a true positive iff IoU>0.5 with an unmatched ground-truth box of the same class (greedy matching by score), and we report $F1 = 2PR/(P + R)$. We also report VOC all-point AP50 per class and mAP50. The threshold for each model is chosen on validation and applied unchanged to test. 95% confidence intervals use 1,000 image-level bootstrap resamples [27]; the paired test resamples the same images for both models.

4 Implementation and Experiments

Environment. Python 3.11, PyTorch 2, Ultralytics 8.3, timm 1.0, Optuna 3.6, MLflow 2.12, scikit-learn, Albumentations; training on a single NVIDIA T4 (16 GB) (Google Colab/Kaggle). Seeds are fixed (42) and deterministic cuDNN is enabled.

Code organisation. The repository is a Python package (`src/roadvision`) with sub-modules `data`, `models`, `tuning`, `evaluation`, `explain`, `visualization` and `inference`; all settings live in YAML (`configs/`); `scripts/run_pipeline.py` executes the idempotent stages download → prepare → tune → train → evaluate → benchmark → explain → plots → report. Unit tests cover data conversion, metrics and the API contract.

Training settings. YOLOv8s, 640 px, 100 epochs (early stopping, patience 20), batch 16, cosine LR, mosaic closed for the final 10 epochs. RT-DETR-L, 60 epochs, batch 8, lr=10^{-4}. ViT-S/16, 25 epochs, batch 64. Tuned values are loaded automatically from `runs/optuna/*_best.json`.

Exploratory analysis. Fig. 2 shows strong class imbalance and country-dependent class mixes (domain shift), and that most boxes cover less than 1% of the image and are either very wide (transverse) or very tall (longitudinal). Box centres concentrate in the lower half of the frame (the road surface).

Training behaviour and tuning. Fig. 3 plots the loss and validation metrics; Fig. 4 shows the Optuna optimisation history and parameter importance.

Representation and explainability. Fig. 5 projects ViT class-token embeddings of validation/test crops with PCA, t-SNE [25] and UMAP [26]. Well separated clusters indicate that the refiner has learnt discriminative features; overlap between D00 and D10 shows where confusion remains. Attention rollout [23] (Fig. 6, top) shows the ViT attending along the crack path rather than to lane paint. Eigen-CAM [24] on the YOLO neck (bottom) confirms that detector activations sit on the damage.

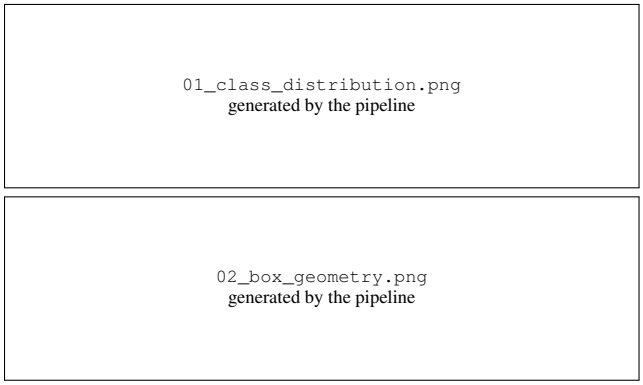


Figure 2: Class distribution by country (top) and box geometry: relative area, aspect ratio and centre heat-map (bottom).

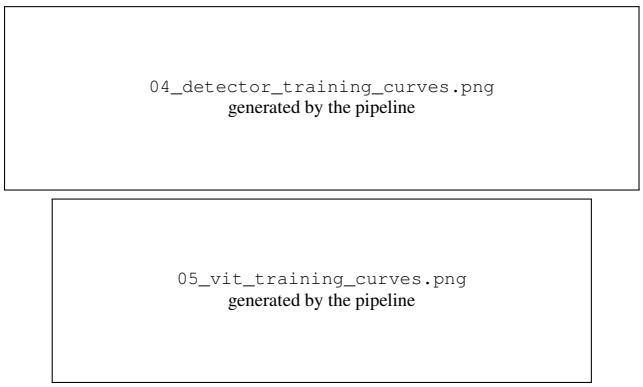


Figure 3: Detector training curves (top) and ViT refiner curves (bottom).

5 Results and Comparative Analysis

Main results. Table **??** compares all models on the held-out test split. The hybrid cascade reaches F1 = – versus –

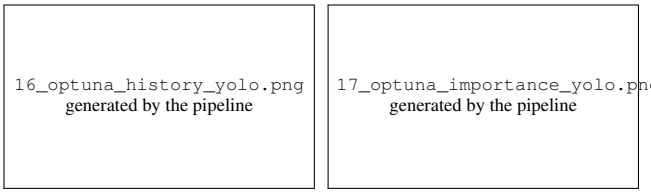


Figure 4: Optuna TPE search for YOLO: optimisation history and fANOVA hyper-parameter importance.

table `main_results` – run the pipeline

for YOLO alone. The paired bootstrap gives ΔF1 = – (95% CI –, one-sided p = –); a CI that excludes zero indicates a statistically reliable gain. The ViT costs extra time per proposal, so throughput falls from – to – FPS, still real-time for dash-cam video sampled at a few frames per second. Per-class AP50 (Table **??**, Fig. 7) shows which damage types benefit most from global attention.

Ablation. Table **??** isolates the cascade components. Removing the background class (setting p_{bg}=0) measures the value of hard-negative mining; α=0 and α=1 give the ViT-only and YOLO-only extremes. Fig. 8 shows the validation F1 as a function of α and of the confidence threshold.

Comparison with the state of the art. Table **??** and Fig. 9 place RoadVision among published results. Only rows that share our protocol (RDD2022-China, random 8:1:1; run `configs/china_811.yaml`) are directly comparable with StripRFNet [6]. Challenge results [3, 5] were computed on hidden test sets whose labels are not public, so they are shown as indicative context only.

Error analysis. The confusion matrix (Fig. 10) uses an explicit background row/column: the last column counts missed damage, the last row counts false alarms. Every hybrid false positive is assigned to one of four types (Fig. 11, left): *background* (no damage nearby; – of FPs), *classification* (right place, wrong crack type; –), *localisation* (right class, IoU 0.1–0.5) and *duplicate*. Recall by object size (right) shows that small, thin cracks remain the main source of misses, which motivates tiling or higher input resolution. Per-country F1 (Fig. 11) quantifies domain shift; the weakest country points to where more labelled data is needed.

6 Deployment

The inference engine (`RoadVisionPredictor`) loads every available model and exposes four modes (yolo, rtdetr, hybrid, ensemble) with the calibrated thresholds. A FastAPI service provides `/predict` (JSON), `/predict/annotated` (PNG), `/predict/batch`, `/health` and `/model-info` with OpenAPI documentation. Inputs are validated by MIME type, magic bytes, a 10 MB size limit, decodability and image-side limits; the mode is an enum and the threshold is bounded, so invalid requests return 4xx errors with explanations. Each response includes a heuristic 0–100 *severity index* $S = \min\left(100, \frac{100}{1.5}\sum_i w_{c_i}\hat{s}_i\sqrt{A_i/A}\right)$ with weights 1/1/2/3 for D00/D10/D20/D40, mapped to Good/Fair/Poor/Critical to rank roads for maintenance. A Gradio UI is mounted at `/ui`. The same Docker image (Python 3.11-slim, CPU PyTorch, non-root user, health-check) runs locally and as a Hugging Face Docker Space; weights are pulled at start-up from a Hugging Face model repository.

table `per_class` – run the pipeline

table `ablation` – run the pipeline

7 Conclusion and Future Scope

RoadVision shows that a lightweight cascade—a fast CNN detector that proposes and a Vision Transformer that verifies—can address the two dominant error modes in road-damage detection, crack-type confusion and background false alarms, without the cost of a multi-model challenge ensemble. Calibrating on validation only, bootstrap confidence intervals and a fixed CRDDC-style protocol make the comparison with baselines reliable, and the containerised API makes the model usable in practice.

Deployment considerations. The ViT adds latency proportional to the number of proposals; batching crops, INT8/ONNX export and capping proposals keep it real-time on edge GPUs. Thresholds should be re-calibrated when the camera type or country changes, and GPS tags are needed to turn detections into maintenance maps.

Future work. (i) Tiled inference and higher resolution for small cracks and the Norway subset; (ii) domain adaptation across countries; (iii) instance segmentation for crack length and pothole area estimation; (iv) temporal tracking across video frames for de-duplication; (v) knowledge distillation of the ViT into the detector head for single-stage speed.

References

[1] H. Maeda, Y. Sekimoto, T. Seto, T. Kashiyama, and H. Omata, "Road damage detection and classification using deep neural networks with smartphone images," *Computer-Aided Civil and Infrastructure Engineering*, vol. 33, no. 12, pp. 1127–1141, 2018.

[2] D. Arya, H. Maeda, S. K. Ghosh, D. Toshniwal, A. Mraz, T. Kashiyama, and Y. Sekimoto, "Deep learning-based road damage detection and classification for multiple countries," *Automation in Construction*, vol. 132, Art. no. 103935, 2021.

[3] D. Arya, H. Maeda, S. K. Ghosh, D. Toshniwal, H. Omata, T. Kashiyama, and Y. Sekimoto, "Global road damage detection: State-of-the-art solutions," in *Proc. IEEE Int. Conf. Big Data*, 2020. [Online]. Available: arXiv:2011.08740

[4] D. Arya, H. Maeda, S. K. Ghosh, D. Toshniwal, and Y. Sekimoto, "RDD2022: A multi-national image dataset for automatic road damage detection," arXiv:2209.08538, 2022.

[5] D. Arya, H. Maeda, S. K. Ghosh, D. Toshniwal, H. Omata, T. Kashiyama, and Y. Sekimoto, "Crowdsensing-based road damage detection challenge (CRDDC'2022)," in *Proc. IEEE Int. Conf. Big Data*, 2022. [Online]. Available: arXiv:2211.11362

[6] J. Lin, Y. Qin, S. Gao, Y. Rui, J. Liu, and Y. Lv, "StripRFNet: A strip receptive field and shape-aware network for road damage detection," arXiv:2510.16115, 2025.

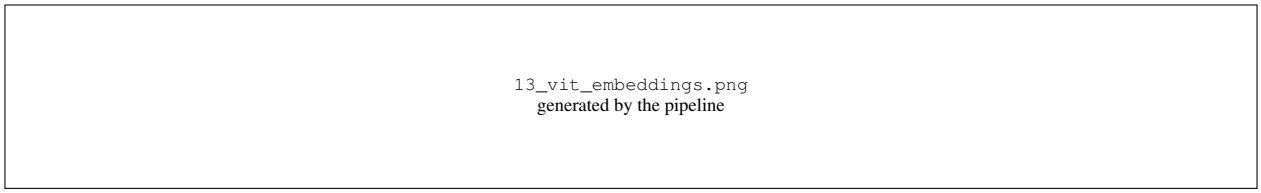
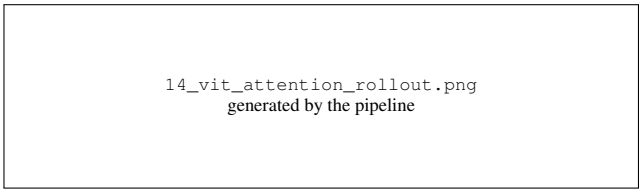


Figure 5: ViT embedding space: PCA cumulative variance and 2-D PCA, t-SNE and UMAP projections coloured by class.



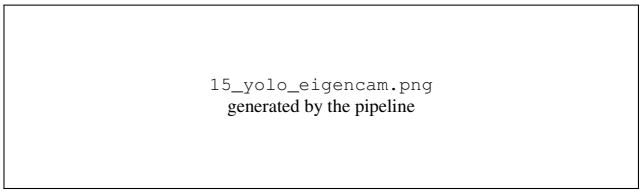


Figure 6: Explainability: ViT attention rollout on test crops (top) and YOLO Eigen-CAM (bottom).

table `benchmark` – run the pipeline

[7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.

[8] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[9] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2015, pp. 91–99.

[10] Z. Cai and N. Vasconcelos, "Cascade R-CNN: Delving into high quality object detection," in *Proc. CVPR*, 2018, pp. 6154–6162.

[11] Z. Liu *et al.*, "Swin Transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021, pp. 10012–10022.

[12] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020, pp. 213–229.

[13] Y. Zhao *et al.*, "DETRs beat YOLOs on real-time object detection," in *Proc. IEEE/CVF CVPR*, 2024, pp. 16965–16974.

[14] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

[15] A. Steiner, A. Kolesnikov, X. Zhai, R. Wightman, J. Uszkoreit, and L. Beyer, "How to train your ViT? Data, augmentation, and regularization in vision transformers," *Trans. Mach. Learn. Res.*, 2022.

[16] R. Wightman, "PyTorch image models," 2019. [Online]. Available: https://github.com/huggingface/pytorch-image-models

[17] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception architecture for computer vision," in *Proc. CVPR*, 2016, pp. 2818–2826.

[18] R. Solovyev, W. Wang, and T. Gabruseva, "Weighted boxes fusion: Ensembling boxes from different object detection models," *Image and Vision Computing*, vol. 107, Art. no. 104117, 2021.

[19] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2019, pp. 2623–2631.

[20] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *Proc. NeurIPS*, 2011, pp. 2546–2554.

[21] M. Zaharia *et al.*, "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Eng. Bull.*, vol. 41, no. 4, pp. 39–45, 2018.

[22] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin, "Albumentations: Fast and flexible image augmentations," *Information*, vol. 11, no. 2, Art. no. 125, 2020.

[23] S. Abnar and W. Zuidema, "Quantifying attention flow in transformers," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2020, pp. 4190–4197.

[24] M. B. Muhammad and M. Yeasin, "Eigen-CAM: Class activation map using principal components," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2020.

[25] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, pp. 2579–2605, 2008.

[26] L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform manifold approximation and projection for dimension reduction," arXiv:1802.03426, 2018.

[27] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.

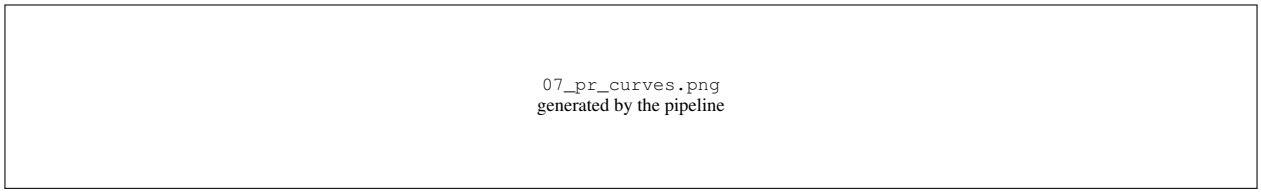


Figure 7: Per-class precision–recall curves on the test split for all models.

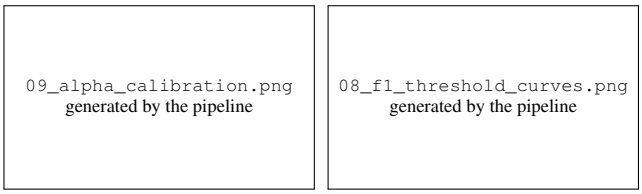


Figure 8: Validation calibration: F1 vs. fusion weight α (left) and vs. confidence threshold (right).

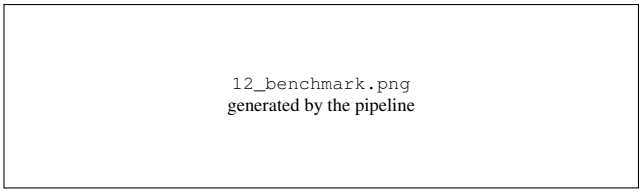


Figure 9: F1 of RoadVision models vs. published detectors (colour indicates comparability).

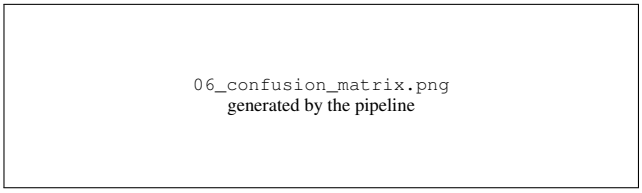


Figure 10: Test confusion matrices (hybrid and YOLO); BG = background / missed.

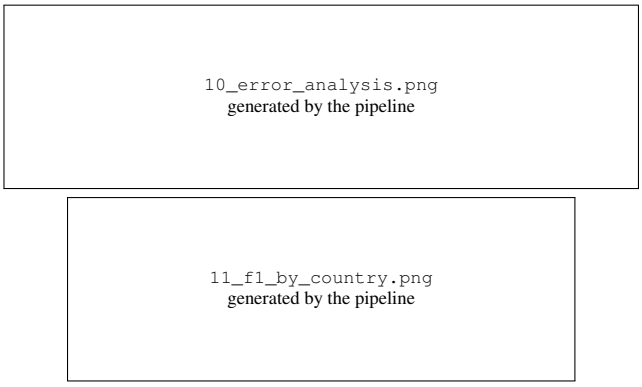


Figure 11: Error taxonomy and recall by size (top); test F1 per country (bottom).